

Software Architecture and Design Specification

Project: Hostel Management System

Version: 1.0

Authors: Pranav Shashikiran Atreya (PES2UG24CS912), Kaushik Balaji (PES2UG24CS220), Kanishk Karthik (PES2UG24CS213), Harshith Manish (PES2UG24CS187)

Team Number: 12

Date: 04-10-2026

Status: Draft for team and course review

Revision History

Version	Date	Authors	Change Summary
1.0	04-10-2026	Pranav Shashikiran Atreya; Kaushik Balaji; Kanishk Karthik; Harshith Manish	Initial architecture and design specification for Hostel Management System

Approvals

Role	Name	Signature and Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document defines the architecture and detailed design of the web-based Hostel Management System (HMS). It provides a shared implementation plan for authentication, student management, hostel and room management, room allocation, transfer and vacation, allocation history, reporting, security controls, persistence, testing and deployment-oriented verification. The design is derived from the approved SRS requirements baseline and the Stage 2 Software Test Plan (STP).

1.2 Scope

The HMS covers authenticated student, warden and administrator access; student record management; hostel and room record management; room availability; room allocation; transfer and vacation; allocation history; occupancy reporting; controlled validation and error handling; and security/audit controls. The architecture is intended for a browser-based client-server application implemented using Python with Django. Physical hostel operations, biometric hardware, external ERP integration, payment processing, food or mess management, attendance management and unrelated hostel services remain outside the project scope.

1.3 Audience

The primary audience is the project development team, QA and validation personnel, security reviewers, course instructors and evaluators, and future maintainers who need a consistent understanding of component boundaries, interfaces, data responsibilities and security controls.

1.4 Definitions

HMS: Hostel Management System. SRS: Software Requirements Specification. STP: Software Test Plan. RTM: Requirements Traceability Matrix. RBAC: Role-Based Access Control. UML: Unified Modeling Language. ORM: Object-Relational Mapping. ADR: Architecture Decision Record. API: Application Programming Interface. UAT: User Acceptance Testing.

2. Document Overview

2.1 How to use this document

Read Section 3 to understand the logical architecture, component boundaries, selected technology, architectural decisions, risks, requirement traceability and security architecture. Read Section 4 for the detailed design, including the persistent data model, sequence flows, API contracts, error handling, logging, monitoring and user-interface guidance. The requirement IDs and test references are retained from the SRS/STP so implementation changes can be reviewed against an existing baseline rather than introducing new requirement identifiers.

2.2 Related Documents

- Software Requirements Specification - Hostel Management System, Version 1.0, dated 05-09-2026.
- Software Test Plan - Hostel Management System, Version 1.0, dated 15-09-2026.
- Requirements Traceability Matrix contained in the SRS and maintained through the STP during execution.
- GitHub repository and Jira Scrum project used for source control, CI/CD and project tracking.

3. Architecture

3.1 Goals & Constraints

The architecture must keep the HMS simple enough for a software-engineering mini-project while maintaining clear separation of responsibilities, testability and security. The selected design is a layered client-server architecture with a modular Django application. The presentation layer handles browser requests and responses; application modules enforce business rules; Django ORM isolates persistence details; and security controls apply at authentication and authorization boundaries.

The architecture is constrained by the SRS technology options: Python with Django or Flask, or PHP with Laravel. The current implementation direction selects Python with Django. The database must be relational and supported by the selected framework. Development uses SQLite, while the deployed/evaluation environment must use the actual relational configuration selected for the project. The STP also requires representative synthetic data, controlled test accounts, and a test environment that mirrors the implemented stack.

The principal SRS acceptance targets remain architectural constraints rather than measured results at design time: 95% of ordinary authenticated page requests should complete within 2 seconds under normal project test load; availability should reach at least 99% during the evaluation period excluding planned maintenance; core workflows should remain usable at a 1280 px viewport without horizontal scrolling; allocation integrity must prevent duplicate active allocations and room over-capacity; and production-like deployments must use HTTPS and secure session/cookie settings.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Students / Residents	Correct personal and hostel information, access to permitted records, clear availability/allocation status, usable web workflows.
Wardens / Hostel Staff	Efficient student/room/allocation management, accurate capacity and occupancy, reliable search and history, controlled permissions.
Administrators	System-wide user/role management, hostel and room configuration, secure administrative operations and auditable changes.
Developers	Modular code boundaries, framework consistency, reproducible environment, clear interfaces and testable business rules.
QA / Validation	Requirement traceability, deterministic test data, unit/integration coverage, controlled errors and security evidence.
Course Evaluators	Runnable demonstrations, visible GitHub activity, working CI/CD, authentic test evidence and alignment to the SRS/STP.

The team responsibilities continue to follow the STP: Pranav Shashikiran Atreya is QA Lead / Team Lead; Kaushik Balaji is Development Lead / Integration Engineer; Kanishk Karthik is Test Engineer / Validation Engineer; and Harshith Manish is Security & System Test Engineer. All team members participate in acceptance evidence and review.

3.3 Component (UML) Diagram

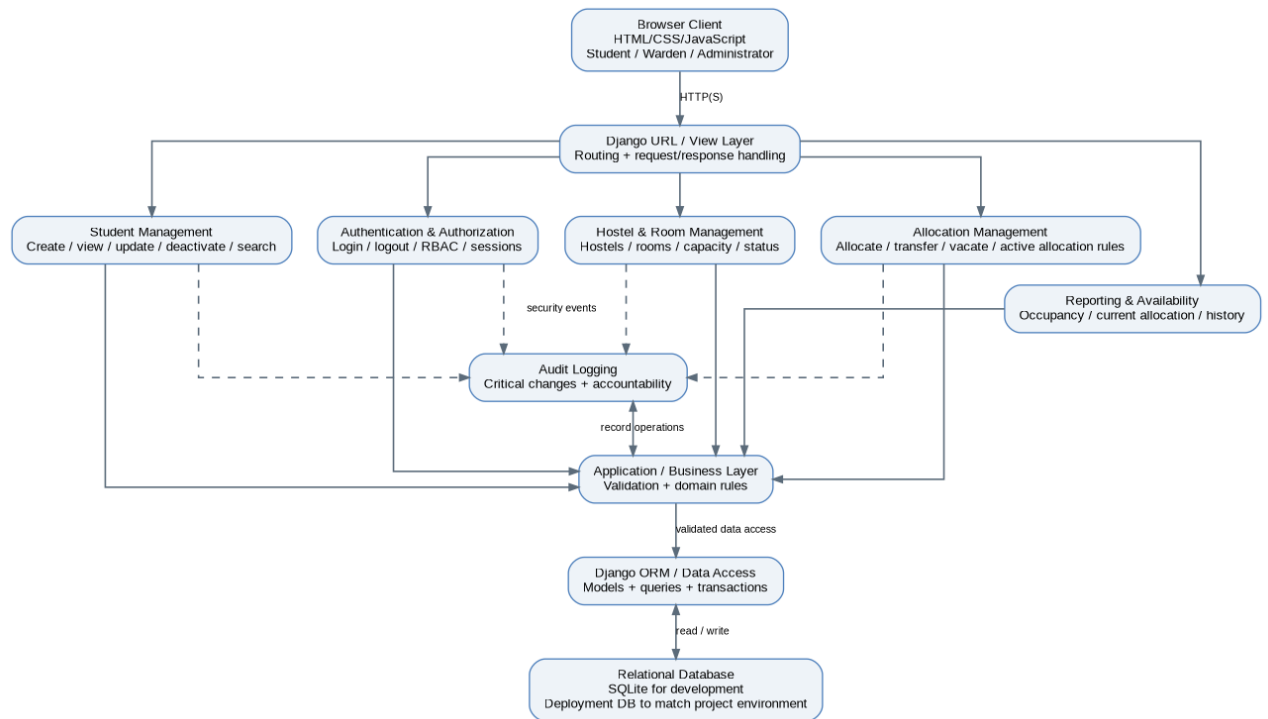


Figure 1. Target HMS components and their dependencies

3.4 Component Descriptions

Component	Responsibility and Boundary
Browser Client	Provides login, role-specific dashboards, student/hostel/room forms, search/filter controls, allocation operations, confirmation messages and controlled error display.
Django URL / View Layer	Routes HTTP requests, validates request context, invokes the appropriate application operation and returns HTML or API responses.
Authentication & Authorization	Uses Django authentication/session facilities, password hashing and role checks. It establishes authenticated identity and denies protected operations when role requirements are not satisfied.
Student Management	Owns student creation, viewing, updating, deactivation, search and filtering. Student identifiers are unique and historical allocation data is preserved.
Hostel & Room Management	Owns hostel and room records, room capacity, room type and status. Rooms belong to a hostel and expose availability based on active allocations.
Allocation Management	Owns allocation, transfer and vacation rules, including active-allocation and capacity validation and preservation of allocation history.
Reporting & Availability	Provides current allocation details, room availability/occupancy calculations, allocation history and authorized occupancy summaries.
Audit Logging	Records critical student, room, allocation and role changes with timestamp, user identity, action type and affected record ID; sensitive credentials are excluded.
Django ORM / Data Access	Maps Python models to relational tables and provides controlled query/transaction access without exposing storage details to presentation code.
Relational Database	Persists users, students, hostels, rooms, allocations and audit records. SQLite is used for development; the deployment database must match the selected evaluation environment.

3.5 Chosen Architecture Pattern and Rationale

The HMS uses a layered client-server architecture implemented as a modular Django application. Django follows an MVT-oriented web structure, while the logical solution separates presentation/routing, application and domain rules, validation/security, and persistence. This is preferred to a microservices architecture because the mini-project scope does not justify independent service deployment, distributed transactions or operational overhead. A modular monolith keeps the development environment reproducible and makes unit and integration testing easier while still giving the code clear boundaries.

The main dependency direction is from browser-facing views toward application/domain modules and then toward the ORM and database. Cross-cutting concerns such as authentication, authorization, audit logging and controlled error handling are invoked at the boundary and should not be bypassed by individual views. Business rules such as unique active allocation and room-capacity enforcement are kept in reusable application logic so the same rules can be tested without relying on a browser session.

ADR 01 - Modular Django application: adopt a layered modular monolith because it fits the project scope, keeps deployment simple and supports incremental integration. The trade-off is that a single application failure affects all modules; this is acceptable for the mini-project and is mitigated through CI checks, structured tests and clear module boundaries.

ADR 02 - Relational persistence through Django ORM: use Django ORM models and transactions rather than scattering raw SQL across views. This preserves consistency for allocation updates and supports a test database without coupling business rules to one database engine.

ADR 03 - SRS-driven module boundaries: align major modules to the SRS feature groups - Authentication, Student Management, Hostel/Room, Allocation, Reporting/History and Audit/Security - so that each requirement group has a corresponding implementation and test location.

3.6 Technology Stack & Data Stores

Layer	Technology and Role
Client	Responsive browser UI using HTML, CSS and JavaScript with Django templates/forms for the core web workflows.
Application	Python + Django selected from the permitted SRS technology options; URL routing, views, forms, authentication and application modules.
Persistence	Django ORM with a relational database. SQLite is used for local development; the evaluation/deployment database follows the selected environment.
Security	Django password hashing, authenticated sessions, role-based authorization, HTTPS in deployed environments, secure cookies/session settings and audit logging.
Testing	Django/Python unit tests, integration tests and manual acceptance/security checks aligned to the STP test references.
CI/CD	GitHub Actions workflow to install dependencies, perform Django checks and execute the automated test suite on push and pull request.
Project Tracking	GitHub for source control and pull requests; Jira Scrum board for Sprint 1 work tracking and traceability.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Allocation logic produces inconsistent occupancy	High	Use atomic validation, explicit active-allocation rules, capacity boundary tests and regression tests after fixes.
Unstable build blocks testing	High	Maintain CI checks on every push/PR; keep a known-good main branch and review failures before merging.
Security configuration differs between local and deployed environments	High	Mirror the selected deployment stack, validate HTTPS/session settings in the deployed configuration and record test evidence.
Insufficient representative test data	Medium	Prepare synthetic students, hostels, rooms, allocations, roles and historical allocations including empty/partial/full capacity states.
Changes break previously working modules	Medium	Use pull requests, review, unit/integration regression checks and Jira status progression before marking work Done.
Framework/database coupling makes deployment difficult	Medium	Keep persistence behind Django ORM and use environment-specific configuration rather than hard-coding deployment assumptions.

3.8 Traceability to Requirements

Each SRS requirement group is mapped to a design component and its existing acceptance-test reference. Grouped IDs are inclusive and retain the identifiers defined by the SRS. This matrix describes design coverage; test execution status remains in the STP/RTM and will be updated after verification.

Requirement IDs	Design Responsibility / Design Reference	Acceptance Test References
HMS-F-001 to 003	Authentication & Authorization / DS-AUTH-01: login, role-based access and logout/session invalidation.	TC-AUTH-01 to 03
HMS-F-004 to 007	Student Management / DS-STU-01: create, view, update/deactivate, search and filter student records.	TC-STU-01 to 04
HMS-F-008 to 011	Hostel & Room + allocation validation / DS-ROOM-01: hostel/room records, availability and capacity rule.	TC-ROOM-01 to 04
HMS-F-012 to 017	Allocation Management / DS-ALLOC-01: allocate, prevent duplicate active allocation, transfer, vacate, current details and history.	TC-ALLOC-01 to 06
HMS-F-018	Reporting & Availability / DS-REPORT-01: occupancy and current allocation summaries.	TC-REPORT-01
HMS-NF-001 to 007	Cross-cutting quality design / DS-NFR-01: performance, availability, integrity, usability, maintainability, robustness and accessibility.	TC-NFR-01 to 07
HMS-SR-001 to 003	Authentication & Session Security / DS-SEC-01: password hashing, role-based authorization and 30-minute inactivity handling.	TC-SEC-01 to 03
HMS-SR-004	Deployment Security / DS-SEC-02: HTTPS and secure session/cookie settings.	TC-SEC-04
HMS-SR-005	Audit Logging / DS-AUDIT-01: critical change logging with safe metadata.	TC-SEC-05

3.9 Security Architecture

The browser is treated as untrusted. Authentication establishes the user identity; authorization is enforced server-side on every protected staff operation; data-access code accepts only validated values; and audit logging records critical changes without exposing secrets. The security architecture implements the two SRS security objectives: confidentiality/least privilege and integrity/accountability.

STRIDE Threat	Design Control
Spoofing	Require valid credentials; use framework password hashing; derive user identity from the authenticated session rather than client-supplied role or user identifiers.
Tampering	Validate form/API input, enforce model constraints and allocation rules, use ORM/database transactions, and reject unauthorized direct URL access.
Repudiation	Record UTC timestamps, authenticated user identity, operation type and affected record ID for critical student, room, allocation and role changes.
Information Disclosure	Enforce role checks; expose only permitted student/allocation data; use HTTPS in deployment; never log passwords, session secrets or other sensitive credentials.
Denial of Service	Use authentication/session controls, bounded inputs, framework request protections and deployment-level connection/resource limits.
Elevation of Privilege	Apply server-side Student/Warden/Administrator authorization checks to every protected function; do not trust client-side role values.

The SRS requires five concrete security controls: one-way password hashing, role-based authorization, 30-minute inactivity handling, HTTPS with secure session/cookie settings, and audit logging. The STP provides corresponding security validation activities TC-SEC-01 through TC-SEC-05.

4. Design

4.1 Design Overview

The detailed design keeps the user interface thin and puts validation and business rules behind reusable application operations. Server-generated identifiers and timestamps remain authoritative. Student, hostel, room and allocation records are persistent entities; allocation history is preserved rather than overwritten. A student may have multiple historical allocation records but at most one active allocation. A hostel contains multiple rooms, and room occupancy is derived from active allocations and configured capacity.

Proposed Record	Core Fields and Invariant
User / Role	Django user identity, password hash, role/group, active state. Username/email is unique; protected functions require authenticated role checks.
Student	student_id, name, contact, course/program, year, status. student_id is unique; deactivation does not delete allocation history.
Hostel	hostel_id, name, block/location, status. hostel_id is unique and a hostel contains many rooms.
Room	room_id, hostel, room_no, room_type, capacity, status. Room number is unique within a hostel; capacity must remain positive.
Allocation	allocation_id, student, room, start/end date, active status. A student has at most one active allocation; transfer closes the old active record and creates the new active record.
AuditLog	timestamp, user, action, entity, record identifier. Critical actions are recorded; password values are never stored in audit data.

4.2 UML Sequence Diagrams

Sequence 1 - Login and role-protected access. The target flow verifies credentials, establishes an authenticated session and performs authorization checks before returning protected data. Invalid credentials or unauthorized roles result in controlled rejection.

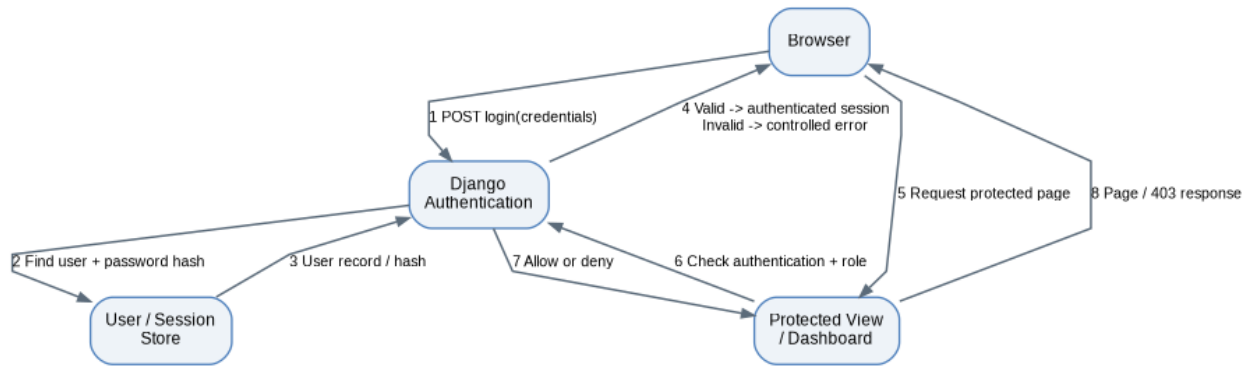


Figure 2. Login and authenticated access sequence

The login flow corresponds to HMS-F-001 to HMS-F-003 and security requirements HMS-SR-001 to HMS-SR-003. The server, not the browser, remains authoritative for identity and permissions.

4.2 UML Sequence Diagrams (continued)

Sequence 2 - Room allocation. The allocation operation first confirms that the student is active and that the target room is available within capacity. Only after all validations succeed is an active allocation created and the critical action recorded. A rejected operation produces no partial allocation state.

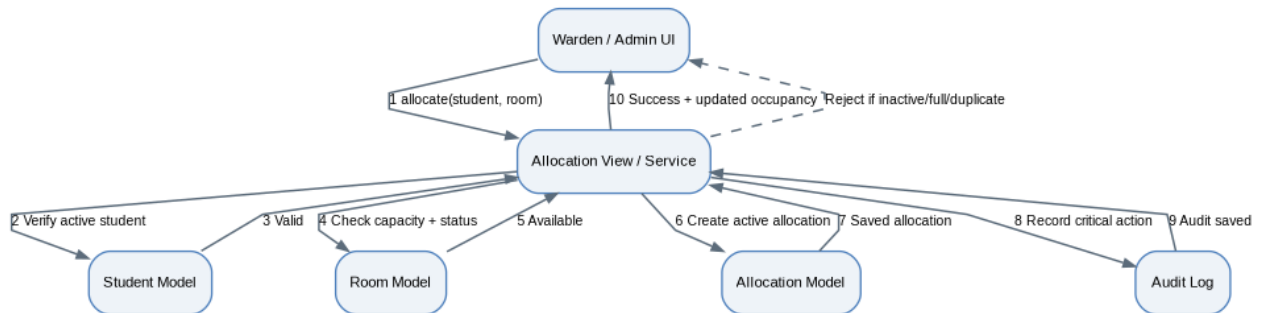


Figure 3. Room allocation and audit sequence

This flow covers the dependencies among student status, room capacity, allocation state and auditability. It supports HMS-F-011 to HMS-F-018 and the integrity/security checks represented by TC-ROOM-04, TC-ALLOC-01 to 06, TC-REPORT-01, TC-NFR-03 and TC-SEC-05.

4.3 API Design

The core UI may use Django-rendered HTML, while the application exposes a small, conventional HTTP API surface for integration, automated tests and future clients. The API is authenticated where the corresponding SRS function is protected. Payloads are validated server-side and errors use consistent status codes and safe messages.

Operation	Request	Success	Errors
POST /api/auth/login	{username, password}	200 {user, session}; authenticated session established.	401 invalid credentials; 429 rate/operational limit if configured.

Operation	Request	Success	Errors
POST /api/auth/logout	Authenticated session	204 / success after session invalidation.	401 unauthenticated/expired session.
GET /api/students?q=...	Authenticated staff request	200 list of matching permitted students.	401 unauthenticated; 403 insufficient role.
POST /api/students	{student_id, name, contact, course_program, year, status}	201 created Student.	400 validation error; 409 duplicate student_id; 403 unauthorized role.

Student management API contracts should remain consistent with the corresponding HTML form operations. The client must never supply server-authoritative identity or permission state in a way that bypasses authentication/authorization checks.

4.3 API Design (continued)

Operation	Request / Parameters	Success	Errors
GET /api/hostels	Authenticated staff request	200 hostel list with id, name, block/location and status.	401 unauthenticated; 403 insufficient role.
POST /api/hostels	{hostel_id, name, block_location, status}	201 hostel record.	400 invalid input; 409 duplicate hostel_id; 403 unauthorized.
GET /api/rooms?hostel_id=...	Authenticated staff/student where permitted	200 room list with capacity, status and derived availability.	401/403 as applicable; 404 hostel not found.
POST /api/rooms	{hostel_id, room_no, room_type, capacity, status}	201 room record.	400 invalid input; 404 hostel not found; 409 duplicate room identifier.
POST /api/allocations	{student_id, room_id, start_date}	201 active allocation.	400 validation; 403 unauthorized; 409 student already allocated or room full; 404 record not found.
POST /api/allocations/{id}/transfer	{target_room_id}	200 new active allocation; previous allocation inactive.	400 invalid transfer; 403 unauthorized; 404 allocation/room missing; 409 target room unavailable/full.
POST /api/allocations/{id}/vacate	{end_date}	200 allocation marked inactive.	400 invalid date/state; 403 unauthorized; 404 allocation missing.
GET /api/reports/occupancy	Authenticated Warden/Administrator	200 occupancy and allocation counts grouped by hostel/room.	401 unauthenticated; 403 insufficient role.

Common result convention: successful responses return the requested data; validation/permission failures use controlled messages; internal exceptions are not exposed. When an operation changes multiple related records, the design uses an atomic transaction boundary so a failed validation does not leave partial allocation state.

4.4 Error Handling, Logging & Monitoring

Errors are separated into validation, authentication, authorization, not-found, conflict and internal categories. A stable set of application-level codes is recommended: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT` and `INTERNAL_ERROR`. Production-like deployments must not expose raw debug tracebacks to end users.

Area	Design
Validation	Reject missing/invalid fields, duplicate identifiers, invalid dates, non-positive capacity and unsupported state transitions before persistence.
Authorization	Check authentication and role before protected actions; direct URL/API access is subject to the same authorization rules as the UI.
Consistency	Perform allocation/transfer/vacation changes atomically and re-check active allocation and room capacity conditions at write time.
Logging	Record UTC timestamp, operation, safe error code, user identity where appropriate, affected resource identifier and relevant duration; exclude passwords, session secrets and sensitive student details not required for diagnostics.
Monitoring	Track test/production-like application errors, authentication failures, allocation conflicts, response-time measurements, CI status and database failures.
Recovery	Return controlled errors, preserve safe client state where practical, and rerun affected regression tests after defect fixes.

4.5 UX Design

The HMS is designed as a responsive browser application with role-aware navigation. Students see only permitted personal and allocation information; Warden and Administrator users see the management functions allowed by their roles. Core screens include login, dashboard, student management, hostel list, room list, room availability, allocation controls, allocation history and occupancy summary.

- Use clear field labels and inline validation for student, hostel, room and allocation forms.
- Provide search and filter controls for student and room lists and clear empty/no-match states.
- Display room status as vacant, partially occupied or full based on active allocation data.
- Use confirmation messages after create/update/allocate/transfer/vacate actions and controlled messages after failures.
- Maintain keyboard-accessible controls, visible focus, readable contrast and responsive layout; core workflows should not horizontally scroll at 1280 px.
- Keep security-sensitive information out of the UI except where explicitly permitted by role and requirement.

4.6 Open Issues & Next Steps

- Confirm the final deployment database and hosting configuration while keeping the local/test setup reproducible.
- Complete authentication/RBAC integration across all protected views and API operations and verify the 30-minute inactivity requirement in the deployed configuration.
- Complete allocation, transfer and vacation transactions and add concurrency-aware capacity validation before final acceptance.
- Expand automated tests so every SRS requirement has at least one test reference, then update the RTM from N to P/A after verification.
- Complete performance, availability, accessibility and security validation using the STP tools and evidence expectations.
- Keep the Jira Sprint history, GitHub branches/commits/PRs and GitHub Actions runs synchronized with implementation progress.

5. Appendices

5.1 Glossary

Term	Definition
HMS	Hostel Management System.
SRS	Software Requirements Specification; the requirements baseline for the project.
STP	Software Test Plan; the Stage 2 test-planning and test-control document.
RTM	Requirements Traceability Matrix; mapping from SRS requirements to design/test references and verification status.
RBAC	Role-Based Access Control.
MVT	Model-View-Template structure commonly used by Django web applications.
ORM	Object-Relational Mapping; maps application models to relational database records.
ADR	Architecture Decision Record.
UAT	User Acceptance Testing.
CI/CD	Continuous Integration / Continuous Delivery or Deployment practices used to automate build/check/test workflows.
STRIDE	Threat-modeling categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege.

5.2 References

- Hostel Management System - Software Requirements Specification, Version 1.0, Team 12, 05-09-2026.
- Hostel Management System - Software Test Plan, Version 1.0, Team 12, 15-09-2026.
- IEEE 42010 - Architecture description concepts and requirements, used as guidance for architecture documentation structure.
- OWASP application security guidance for secure authentication, authorization, input validation and error handling.
- Project Jira Scrumboard and GitHub repository documentation maintained by Team 12.

5.3 Tools

- Python 3.x and Django for application development.
- SQLite for local development with a relational database configuration matching the evaluation environment.
- Django automated test framework for unit and integration tests.
- Git/GitHub for version control and collaboration; GitHub Actions for CI/CD; Jira for Scrum tracking; PlantUML/diagrams.net/draw.io for UML diagrams.